Foreword

In the autumn of 1969, a handful of graduate students at UCLA and a few sister sites were wiring together the first four nodes of a network called the ARPANET. Nobody involved knew yet whether any of it would hold. Among the students was a young researcher named Jon Postel. The group needed a way to write down, informally and without waiting for anyone's official blessing, how these machines were supposed to behave toward each other. They called the notes Requests for Comments. RFCs.

An RFC specified requirements: what a message must contain, what a receiver must check, what counts as an error, and what happens when one occurs. It left the actual software design open, to be decided by whoever was building the machine in front of them.

Postel took on the job of collecting and editing those notes, a role that later became known as the RFC editor, and he held it for almost three decades. It's the reason this book exists in its current form, though most of the people who will read it have never heard his name.

A specification that outlived every implementation

By 1981, two of the documents Postel edited, RFC 791 and RFC 793, set down the rules for the Internet Protocol and the Transmission Control Protocol, the pair spoken as TCP/IP: a description of what any implementation had to do to talk to any other implementation, detailed enough that two programmers who never met could each build a version and get the two talking, working from the document rather than from each other.

Those two documents are, at the time of this writing, older than most of the engineers who will read this book. And they have outlived every implementation ever written against them, several times over. COBOL mainframes ran an early implementation. Unix systems ran another. When personal computers arrived, they got their own. Windows, Linux, and macOS each shipped a version. Cloud platforms virtualized it. Containers packaged it again. Embedded devices squeezed it into a few kilobytes of flash memory. None of those implementations exists today in the form it first shipped. The rules they were all built to satisfy do.

> **The specification outlived every machine built to satisfy it, including machines that did not exist when the specification was written.**

RFC 791 and RFC 793 persisted for a reason that goes beyond networking. They described a stable thing, what has to happen, and left an unstable thing, how any one system happens to build it, entirely open. Technology kept changing underneath them for exactly that reason, without ever touching what they said.

The same discipline, at the scale of one organization

This book takes that fifty-year-old lesson and applies it somewhere much smaller: a single team, a single codebase, a single organization's business rules. Most of those organizations still do the opposite of what the internet's protocol designers did. They let the source code carry the business knowledge. They lose that knowledge the moment the language or the platform changes. They rediscover it, expensively, whenever a system finally gets rewritten and someone has to work out what the old one actually did.

AI coding assistants made that problem visible almost overnight. Ask an assistant to build a feature from a short description, and it will produce something. If nobody wrote down the actual rule, the assistant invents a plausible one, filling the gap with what businesses of that kind usually do rather than what this one decided. The rest of this book calls that the difference between asking AI to guess and asking it to implement a decision someone already made.

The methodology here is Simon Martinelli's, and it continues a lineage that already runs deep: structured analysis, use cases, domain-driven design, Agile practice, architecture decision records. Each of those contributed a piece of the same idea, that a system's rules deserve a durable form separate from whatever code happens to implement them today. What changes now is the economics. When implementing a feature took weeks, the hours spent writing a precise specification first were easy to skip. When an AI assistant can implement that same feature in minutes, skipping the specification stops being a shortcut and starts being the most expensive step in the process.

The payoff doesn't show up on day one. It shows up the first time a rule changes and nobody has to hunt through old code to find where it lived. It shows up again when a new engineer reads the specification instead of reverse-engineering the system from its side effects. It shows up a third time when the underlying platform gets replaced and the business rules simply move to the new one, the way TCP/IP moved from mainframes to phones without anyone renegotiating what a packet is.

One claim, three kinds of reader

The chapters ahead are written for whoever is closest to the work: the engineer writing a use case for the first time this week, the architect trying to keep a dozen AI-assisted teams from drifting apart, the manager deciding what "done" should mean once implementation is no longer the slow part. Each of them will find a different amount of technical detail useful, and the book is arranged so each can take only what they need.

What ties their chapters together is the same claim Postel's generation made about the network, applied to a single company instead of the whole internet: write down what the system must do, keep that document separate from how any particular platform happens to build it, and the document will outlast the platform. RFC 791 did not know it would still be read in an era of phones, containers, and machine-written code. Your specification does not need to know what replaces your current stack either. It only has to say, precisely, what has to be true regardless.

Somewhere a copy of RFC 793 is still sitting in a standards archive, unglamorous and unchanged, outlasting every system ever built to satisfy it. This book is about writing your organization's version of that document, and about what changes, in how you work and what AI can do for you, once you have.